LESSON 3

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Push the Button

Give your ESP32 an input, so it can react to you.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- The difference between **input** and **output**
- How to read a **button** with `digitalRead`
- How to make decisions with **if / else**
- How to remember things with a **bool** variable
- How a **tilt switch** can sense shaking

SESSION PLAN

0:00	Warm-up: inputs and outputs all around us
0:10	Add a button to your circuit
0:20	Sketch 1: **L03_Button**
0:35	Sketch 2: **L03_Toggle** (on/off switch)
0:55	Short break
1:00	Sketch 3: **L03_Counter** (count presses)
1:20	Sketch 4: **L03_ShakeAlarm** (tilt switch)
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ Your traffic light LEDs from Lesson 2 (red 25, yellow 26, green 27)
- ☐ 2 push buttons
- ☐ 1 tilt ball switch
- ☐ 4 jumper wires

1. Inputs and outputs

So far the ESP32 has only **talked**: it switched LEDs on and off. Those are **outputs**. Today it will **listen**. A button is an **input**: the ESP32 checks whether it is pressed, then decides what to do.

> **Spot the inputs**
>
> Think of a TV remote, a game controller or a microwave. Name two **inputs** and two **outputs**.
>
> _____
>
> _____

2. Add a button

A push button has 4 legs. Inside, the legs are joined in **pairs**. Pressing the button connects the two pairs together.

> **The diagonal trick**
>
> Put the button across the middle gap of a breadboard. Then use two legs that are **diagonally opposite**. That way you always get one leg from each pair, whichever way round the button is.

Keep your traffic light LEDs from Lesson 2 in place, then add:

✓	From	To
☐	Push button	across the middle gap of a breadboard
☐	ESP32 pin **13**	one leg of the button
☐	the **diagonally opposite** leg	**–** rail (GND)

That's it: no resistor needed! Read on to find out why.

3. Sketch 1: Light while pressed

File → New Sketch, type this in, save it as **L03_Button**, and upload it. Hold the button down: the yellow LED should light up.

Type this in → save it as **L03_Button** — 18 lines

```
// Lesson 3: Press a button to light an LED
// Button on pin 13 (other leg to GND), yellow LED on 26.

const int BUTTON = 13;
const int LED = 26;

void setup() {
  pinMode(BUTTON, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
}

void loop() {
  if (digitalRead(BUTTON) == LOW) {
    digitalWrite(LED, HIGH);   // pressed
  } else {
    digitalWrite(LED, LOW);    // not pressed
  }
}
```

How it works

Code	What it means
`pinMode(BUTTON, INPUT_PULLUP);`	Pin 13 is an **input**, and the ESP32 gently pulls it up to **HIGH**
`digitalRead(BUTTON)`	Read the pin: `HIGH` or `LOW`
`== LOW`	**Is it equal to** LOW? Note the **two** equals signs!
`if ( … ) { … } else { … }`	If the test is true, do the first block. Otherwise, do the `else` block

Big idea: pull-up means pressed = LOW

With `INPUT_PULLUP`, the pin reads **HIGH** when nothing is happening. Pressing the button connects the pin to GND, so it reads **LOW**. It feels backwards, but it means we don't need an extra resistor.

`=` is not `==`

One `=` **puts** a value in a variable: `count = 5;`. Two `==` **compare** two values: `if (count == 5)`. Mixing them up is the most common bug in all of programming!

4. Sketch 2: An on/off switch

Holding the button down is tiring. Let's make it work like a light switch: press once for **on**, press again for **off**. To do that, the ESP32 has to **remember** whether the light is on.

File → New Sketch, save it as **L03_Toggle**.

Type this in → save it as **L03_Toggle** — 25 lines

```
// Lesson 3: Press once for on, press again for off
// Same wiring as L03_Button.

const int BUTTON = 13;
const int LED = 26;

bool ledOn = false;
bool wasPressed = false;

void setup() {
  pinMode(BUTTON, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
}

void loop() {
  bool pressed = digitalRead(BUTTON) == LOW;

  if (pressed && !wasPressed) {
    ledOn = !ledOn;
    digitalWrite(LED, ledOn);
  }

  wasPressed = pressed;
  delay(20);   // ignore the bouncing contacts
}
```

Code	What it means
`bool ledOn = false;`	A **bool** holds only `true` or `false`. It is our memory of the LED
`pressed && !wasPressed`	`&&` means **and**. `!` means **not**. So: pressed now, and *not* pressed last time
`ledOn = !ledOn;`	Flip it: `true` becomes `false`, `false` becomes `true`
`wasPressed = pressed;`	Remember the button for next time round the loop

Big idea: the moment of pressing

`loop()` runs thousands of times a second. If we flipped the LED every time the button *is* down, it would flicker crazily. We only flip at the moment the button **changes** from up to down.

Try it

Delete the `delay(20);` line and upload. Press the button many times. Does it sometimes miss or flip twice? Buttons **bounce**: the metal contacts rattle for a few milliseconds. The short delay makes the ESP32 ignore the rattle. Put the line back!

5. Sketch 3: Count the presses

Add a second button, the same way as the first:

✓	From	To
☐	ESP32 pin **14**	one leg of the second button
☐	the **diagonally opposite** leg	**–** rail (GND)

File → **New Sketch**, save it as **L03_Counter**. Upload, then open the **Serial Monitor** (115200 baud).

Button 13 adds one; button 14 resets the count to zero. The `delay(300)` after a reset stops it printing *Reset to 0!* hundreds of times while you hold the button.

Type this in → save it as **L03_Counter** — 33 lines

```
// Lesson 3: Count button presses
// Count button on 13, reset button on 14 (both to GND).

const int COUNT_BTN = 13;
const int RESET_BTN = 14;

int count = 0;
bool wasPressed = false;

void setup() {
  Serial.begin(115200);
  pinMode(COUNT_BTN, INPUT_PULLUP);
  pinMode(RESET_BTN, INPUT_PULLUP);
}

void loop() {
```

```
  bool pressed = digitalRead(COUNT_BTN) == LOW;

  if (pressed && !wasPressed) {
    count = count + 1;
    Serial.print("Presses: ");
    Serial.println(count);
  }
  wasPressed = pressed;

  if (digitalRead(RESET_BTN) == LOW) {
    count = 0;
    Serial.println("Reset to 0!");
    delay(300);
  }

  delay(20);
}
```

> **Try it**
>
> How many presses can you do in 10 seconds? Get a friend to time you!

6. Sketch 4: Shake alarm

A **tilt ball switch** has a tiny metal ball inside. When it's upright, the ball touches the contacts and the switch is **closed**. Tip it or shake it, and the ball rolls away: the switch **opens**. It works just like a button that is pressed by gravity!

Take out the second button (pin 14) and put the tilt switch in its place:

✓	From	To
☐	ESP32 pin **14**	one leg of the tilt switch (either way round)
☐	other leg of the tilt switch	**–** rail (GND)

File → New Sketch, save it as **L03_ShakeAlarm**.

Type this in → save it as **L03_ShakeAlarm** — 31 lines

```
// Lesson 3: Shake alarm
// Tilt switch on pin 14 (other leg to GND), red LED on 25.

const int TILT = 14;
const int ALARM_LED = 25;

int lastTilt;
int shakes = 0;

void setup() {
  Serial.begin(115200);
  pinMode(TILT, INPUT_PULLUP);
```

```
  pinMode(ALARM_LED, OUTPUT);
  lastTilt = digitalRead(TILT);
}

void loop() {
  int tilt = digitalRead(TILT);

  if (tilt != lastTilt) {
    shakes = shakes + 1;
    Serial.print("Shake detected! Total: ");
    Serial.println(shakes);

    digitalWrite(ALARM_LED, HIGH);
    delay(1000);
    digitalWrite(ALARM_LED, LOW);

    lastTilt = digitalRead(TILT);
  }
}
```

Code	What it means
`int lastTilt;`	Remember what the switch was last time
`tilt != lastTilt`	`!=` means **is not equal to**. Something changed: it was shaken!
`lastTilt = digitalRead(TILT);`	After the alarm, read it again so we start fresh

7. Challenges

Challenge 1: Two-button control

Use button 13 to turn the green LED **on** and button 14 to turn it **off**.

Challenge 2: Pedestrian crossing

Combine Lesson 2 and today. Cars get a green light, until somebody presses the button. Then the light goes yellow → red for 5 seconds → back to green. *Hint: copy the* `lights()` *function from L02_LightFunction.*

Challenge 3: Reaction game for two

The yellow LED switches on after a random wait (`delay(random(2000, 6000));`). Player 1 has button 13, player 2 has button 14. Whoever presses first wins, and their LED (red or green) lights up.

Bonus: Secret knock

The green LED only lights up if you press button 13 exactly **3 times** and then button 14.

8. Quiz

Check what you learned

1. With `INPUT_PULLUP`, what does `digitalRead` give when the button is pressed?

2. What is the difference between `=` and `==`?

3. What values can a `bool` variable hold?

Tidy up: unplug the USB cable. Put the tilt switch back in the kit. Keep the LEDs and the buttons on 13 and 14 for later lessons.